

# LECTURE - 24

wednesday  
29/4/26

## Mapping Union Relations into Tables:

- Always make separate JOIN Tables for each entity.
- If there isn't a key of "Subclass" (single entity) then we should make one ourselves.

## Transactions:

ACID: Atomic (whole transaction or none), Consistent (DB constant consistent & preserved), Isolated (every user thinks only they're processing at that time), Durable (Effect of process survive crash).

⇒ Dirty Read: T1 & T2 dealing with cell A of database.

T1 do  $100 + 50$  and writes 150  $\therefore (A=150)$ .

Then at same time T2 do  $A - 30$   $\therefore (A=120)$  but now

T1 fails instead of being committed. Therefore T2 used a value from A after T1 just wrote data & didn't committed/succeeded which caused T2 to have totally wrong value.

Read: Select  
write: Update

OR write(X) after write(X) without commit.

→ If "Read" is first & then "write" before commit, it causes LOST UPDATE.

\* Whenever there is 'Read(X)' right after 'write(Y)' with no 'commit/success' b/w them; it's dirty read.

Read & write should be of same cell/table

⇒ Non-Repeatable Read: violates Isolation to some extent

• We wanna read(select) a cell again and again; the value should be same BUT since someone did update that cell in b/w two reads; ∴ both reads have different values.

⇒ Phantom: Almost same as Non-repeatable read; when par there was "UPDATE" b/w two reads, ya then par "INSERT" hogi b/w two reads. ∴ 1<sup>st</sup> read may have 3 rows but 2<sup>nd</sup> read have 10 rows.

NOTE: For above 3 problems; we can compromise b/w these three and ISOLATION ∴ we'll tell DB which problem could be allowed & which couldn't be.

- To not allow dirty reads at all; we say it Serialization.
- Strict Schedule doesn't allow dirty read & lost update.

Sa; r<sub>1</sub>(X); r<sub>2</sub>(X); w<sub>1</sub>(X); r<sub>1</sub>(Y); w<sub>2</sub>(X); w<sub>1</sub>(Y)

Sa

| T1                 | T2                 |
|--------------------|--------------------|
| r <sub>1</sub> (X) |                    |
|                    | r <sub>2</sub> (X) |
| w <sub>1</sub> (X) |                    |
| r <sub>1</sub> (Y) |                    |
|                    | w <sub>2</sub> (X) |
| w <sub>1</sub> (Y) |                    |

⇒ Schedules:

⇒ Recovery. On the basis where "commit" is in the Transaction.

⇒ RollBack

⇒ Abort (commit)

⇒ Strict schedule:

~~Serial~~ ⇒ Cascadeless

